STREAMRESOURCE · PRODUCTION READINESS GUIDE

From Prototype to Production

The Angular Agent Readiness Guide

cacheplane.io · 2026

Contents

01 Streaming State Management

02 Thread Persistence

03 Tool-Call Rendering

04 Human Approval Flows

05 Generative UI

06 Deterministic Testing

CHAPTER 1

Streaming State Management

Overview

When you move from prototype to production, the requirements change fundamentally. What worked in a demo — direct API calls, synchronous state, manual zone management — falls apart at scale.

The Signals-Native Approach

StreamResource provides a signals-native approach that eliminates the boilerplate:

```
@Component({...})
export class ChatComponent {
  chat = streamResource<{ messages: BaseMessage[] }>({
    assistantId: 'chat_agent',
  });
}
```

Production Checklist

- Are your message signals OnPush-compatible?
- Is isStreaming() driving your loading UI without polling?
- Are you avoiding manual zone patching?

CHAPTER 2

Thread Persistence

Overview

Demos work with ephemeral state. Production agents need conversation history that survives page refreshes, tab switches, and navigation — wired to LangGraph's MemorySaver backend.

The threadId Pattern

```
provideStreamResource({
  apiUrl: 'http://localhost:2024',
  threadId: signal(localStorage.getItem('threadId')),
  onThreadId: (id) => localStorage.setItem('threadId', id),
})
```

Production Checklist

- Does your agent UI resume threads correctly after a browser refresh?
- Can users switch between conversations?
- Is thread state scoped correctly per user?

CHAPTER 3

Tool-Call Rendering

Overview

LangGraph agents invoke tools mid-stream. The UI needs to show tool execution state in real time — steps appearing as the tool runs, a final result, and collapsible history.

Progressive Disclosure

Production Checklist

- Do your tool call cards handle partial step state during streaming?
- Is tool history collapsible after completion?
- Can you distinguish pending vs. completed tool calls?

CHAPTER 4

Human Approval Flows

Overview

Production agents that take consequential actions must pause for human approval before proceeding. This requires a tight loop between LangGraph's interrupt() primitive and Angular UI.

The Interrupt Pattern

```
// In your component
approved = computed(() => this.chat.interrupt()?.approved ?? false);
onApprove() {
  this.chat.submit(null, { command: { resume: true } });
}
```

Production Checklist

- Can your agent UI recover gracefully if a user cancels an interrupt?
- Are approve/edit/cancel actions clearly mapped to resume commands?
- Is interrupt state persisted across refreshes?

CHAPTER 5

Generative UI

Overview

The most advanced production agents emit structured UI specs — not just text. A data analysis agent might render a live table. A booking agent might render a reservation form.

The Registry Pattern

```
defineAngularRegistry({
  'data-table': DataTableComponent,
  'booking-form': BookingFormComponent,
  'chart-widget': ChartWidgetComponent,
});
```

Production Checklist

- Can your agent emit UI components without tight coupling to the frontend codebase?
- Does JSON patch streaming enable progressive UI updates?
- Is your component registry decoupled from agent logic?

CHAPTER 6

Deterministic Testing

Overview

Agent UIs are notoriously hard to test because they depend on live LLM responses. Flaky tests, slow CI, and inability to reproduce edge cases are the main reasons agent UIs ship with low confidence.

The MockStreamTransport Approach

```
const ref = createMockStreamResourceRef();
ref.messages.set([{ role: 'assistant', content: 'Hello' }]);
ref.isStreaming.set(false);
expect(fixture.nativeElement.querySelector('.message').textContent)
  .toBe('Hello');
```

Production Checklist

- Do your agent component tests run offline and complete in under 100ms each?
- Are streaming, interrupts, tool calls, and generative UI all tested in isolation?
- Is MockStreamTransport used instead of mocking streamResource() itself?